

HANDS-ON WORKSHOP • 75 MINUTES

From a folder to a live website

You will change two lines in a file, press save, and watch a robot put your work on the internet.

Amaxst

1

Your folder

on your screen

2

GitHub Actions

notices and runs

3

Azure Storage

live on a URL

What you walk out with

By the end of the session, on your own screen.

1

A live URL

A web address anyone can open,
serving files you created.

2

A working pipeline

Push a change, and it publishes itself
in under a minute.

3

The mental model

Enough understanding to read any
deploy pipeline you meet later.

Nothing you build today is thrown away — the same pattern runs real production deployments.

The problem we are solving

Two ways to get a folder from your screen onto a server.

By hand

- Open a portal in a browser
- Find the right place to put things
- Drag the files across, wait
- Try to remember which files changed
- Do it again tomorrow, slightly differently

No record of who did what, or when.

Automated

- Save your change
- ...that is the whole list

Every deploy is recorded, repeatable, and identical. Anyone on the team can trigger one, or see why one failed.

The machine never forgets a step.

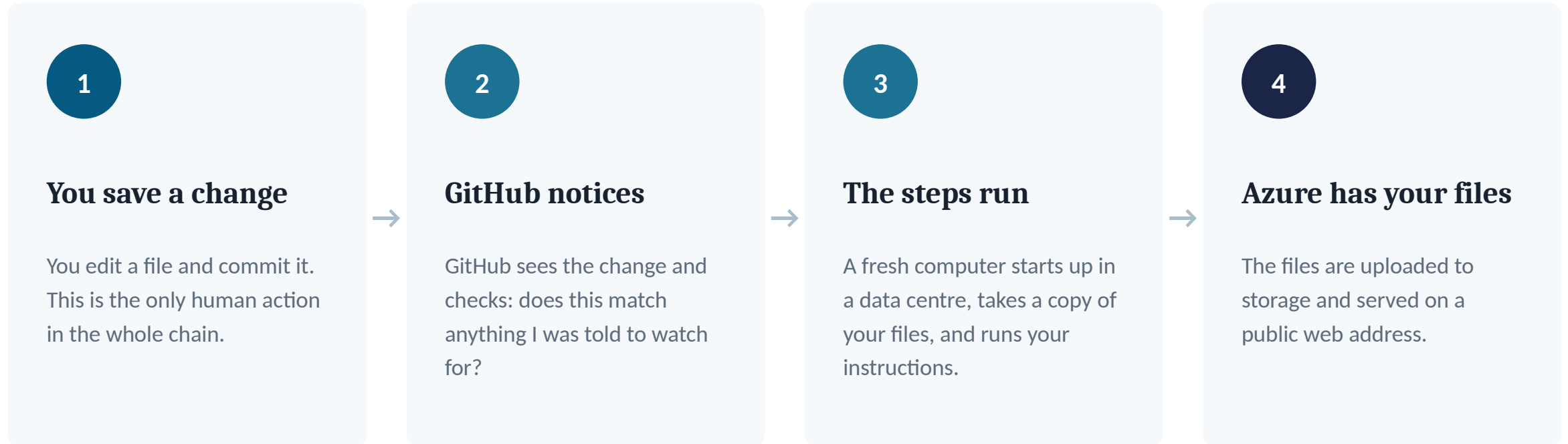
WATCH FIRST, UNDERSTAND SECOND

Let me show you the whole thing

No explanation yet. Just watch what I touch and what happens on its own. Seven minutes.

What just happened

Four steps. Only the first one involved a human.



The whole chain took about 40 seconds, and it will take about 40 seconds every single time.

The instructions live in one file

A workflow file has three parts. You will only touch the first two.

on:

WHEN should this run?

The trigger. A gate — if nothing here matches, nothing happens at all.

env:

WHAT should it work on?

Settings the steps read. Your folder name goes here.

jobs:

HOW should it do it?

The actual checklist of commands. Written for you — leave it alone today.

It is a plain text file called `deploy-to-azure.yml`. Nothing is hidden — you can read the whole thing.

The one idea worth remembering

These are two different mechanisms that happen to need the same value.

The gate

paths:

Decides whether the run happens at all.

Checked before anything starts. If your change is not inside the folder listed here, GitHub does nothing — no run, no log, no failure. Silence is the correct answer.

The instruction

SITE_FOLDER

Decides what gets uploaded.

Read after the run has already started. This is the folder the steps will actually copy to Azure.

Change one and forget the other, and the workflow tells you so in plain English. Expect to do this at least once — most people do.

The only two things you change

Both are marked in the file. Search for “EDIT ME”.

EDIT 1 OF 2

under on: → push: → paths:

FROM

```
- 'site-sample/**'
```

TO

```
- 'site-jason/**'
```

Keep the quotes and the `/**` on the end.

EDIT 2 OF 2

under env:

FROM

```
SITE_FOLDER: site-sample
```

TO

```
SITE_FOLDER: site-jason
```

No quotes, no `/**` here.

Replace `site-sample` with `site-` and your own GitHub username. Same spelling both times, including capital letters.

Where the password lives

The workflow needs permission to write to Azure. That permission is not in the file.

The connection string is a password for the storage account. Anyone holding it can read and overwrite everything in it.

So it is stored in the repository's settings as a secret, not written into the workflow. The file just refers to it by name.

Once saved, nobody can read it back — not even the person who put it there. GitHub also blanks it out of the logs automatically.

It is already set up for you today. You will never see it or type it.

HOW THE FILE REFERS TO IT

env:

AZURE_STORAGE_CONNECTION_STRING:

`${{ secrets.AZURE_STORAGE_CONNECTION_STRING }}`

A name, not a value.

When the password changes, one setting is updated and every workflow keeps working. No code is edited, nothing is redeployed.

HANDS-ON • PARTS 1 AND 2

Your turn

Twenty minutes. The next two slides stay up on the screen — you do not need to keep flipping through the handout.

Part 0 — fork it and set it up

8 minutes

We do these three together. Housekeeping only — shout if you fall behind.

1

Fork the repository

Click `Fork` (top right) → `Create fork`. You now have your own copy of everything.

2

Turn on Actions in your fork

`Actions` tab → click “I understand my workflows, go ahead and enable them.” Forks arrive with Actions switched off.

3

Add the credential to your fork

`Settings` → `Secrets and variables` → `Actions`. Secret: `AZURE_STORAGE_CONNECTION_STRING` . Variable: `AZURE_WEB_ENDPOINT` .

Paste the connection string into the Secrets box and nowhere else

Your fork is public. Anything committed to a file in it is visible to everyone, permanently — even if you delete it afterwards. Never into the workflow file, a comment, or a README.

Part 1 — make your own folder

10 minutes

You are in your own fork now, so work straight on main. No branches needed.

1

Start your folder

Add file → Create new file. In the filename box type: `site-your-username/index.html`

2

Fill it in

Open `site-sample/index.html` in another tab, copy everything, paste it in, then Commit changes.

3

Repeat twice

Do the same for `styles.css` and `data.json` so your folder has all three files.

4

Put your name on it

In your `index.html`, replace “your name here” with your actual name. Commit.

Typing a / in the filename box is how you create a folder in the GitHub website.

Part 2 — point the workflow at your folder

10 minutes

Open `.github/workflows/deploy-to-azure.yml` and click the pencil icon.

1 In the `paths:` list near the top

```
- 'site-sample/**'
```



```
- 'site-yourname/**'
```

Keep the quotes. Keep the `/**`.

2 In the `env:` block below it

```
SITE_FOLDER: site-sample
```



```
SITE_FOLDER: site-yourname
```

No quotes. No `/**`.

Then: Commit changes → Commit directly to main (it is your fork)

The two values must match each other exactly, including capital letters. If they do not, the run fails and tells you which folders it could actually find — read that message, it names the problem.

HANDS-ON • PART 3

Push it and watch

Go to the Actions tab. Your run is probably already there.

Part 3 — read the run

8 minutes

Actions tab → click into your most recent run.

1

Why did this run?

Prints what triggered it, which branch, and your commit message. Confirms GitHub saw you.

2

Work out which folder to deploy

Checks your folder exists and counts the files. This is where a mismatch gets caught.

3

Upload files to Azure Storage

One line per file, with the file type it was uploaded as.

4

Write the run summary

Go back to the run's main page — your live link is at the top.

WHAT THE COLOURS MEAN



Yellow

still running



Green

it worked



Red

it failed — open it and read

Red is normal. Everyone gets one.

Open your live link. Your name should be on a real web page, served from Azure.

Three experiments

The most useful fifteen minutes of the session. One of these is supposed to do nothing at all.

Part 4 — three experiments

15 minutes

Predict what will happen before you do each one.

A

A normal deploy

DO THIS

In your `styles.css`, change the accent colour to `#B8336A` and commit.

EXPECT

The workflow runs. Reload your live page — the colour has changed.

B

The one that does nothing

DO THIS

Create a file in the top level of the repo called `notes.md` with any text. Commit it.

EXPECT

No run appears at all — the change was outside the watched folder.

C

The manual button

DO THIS

Actions → “Deploy folder to Azure Storage” → Run workflow. Pick `main`, leave the box empty.

EXPECT

It deploys again with no code change. Then try a neighbour’s folder name in the box.

Do all three, including B. Especially B.

Why experiment B matters

A system that correctly does nothing is doing real work.

Picture this repository in six months.

It holds far more than the website: documentation, tests, configuration, build scripts, a README somebody edits every week.

Without the paths filter, every single one of those commits would redeploy everybody's work. Twenty times the cost, twenty times the waiting, and no way to tell whose change broke what.

The filter is what keeps a deploy meaning something.

Change inside your folder

A run starts. Your folder deploys.

Change anywhere else

Nothing happens. Correctly.

“Nothing happened” is the hardest failure to diagnose — and the first thing to check is whether anything was supposed to happen at all.

If it breaks

Almost every failure in this workshop is one of these five.

No run appears at all

Either Actions is still disabled in your fork (Part 0, step 2), or the change was outside the watched folder — experiment B.

“Folder ... does not exist”

The two values do not match. The error lists the folders that do exist — compare them letter by letter.

Run is green, page is a 404

Wait ten seconds and hard-reload. Check your URL ends with a slash after your folder name.

Page downloads instead of opening

A file type problem. Rare here, since the workflow sets types explicitly.

“Secret ... is not set”

Not your fault. That is repository setup — put your hand up.

The workflow is written to fail loudly and say what is wrong. Read the red step before asking — it is usually the answer.

Four things to take away

In order of how much they will matter to you later.

1

A pipeline is a trigger plus a checklist

Most of the difficulty in real systems is being precise about the trigger, not about the steps.

2

Scope your automation

Something that fires on everything is usually worse than something that fires on the right things.

3

Keep passwords out of the file

Reference them by name. Rotating a credential should touch one setting, not your code.

4

Always give yourself a manual button

Two lines of setup that will save you on the day something is on fire.

What is different in a real system

Today's setup is honest training wheels. Here is what changes.

TODAY

A stored password

- One connection string, saved as a secret
- Works everywhere, easy to explain
- Grants full access to the whole account
- Has to be rotated by hand

IN PRODUCTION

No stored password at all

- GitHub proves who it is to Azure directly
- Nothing secret is ever stored anywhere
- Permissions scoped to one container
- Nothing to rotate, nothing to leak

Same workflow file. Only the authentication step changes.

YOU BUILT A DEPLOYMENT PIPELINE

That is the whole trick

Everything more sophisticated than what you did today is a variation on it: a trigger, a checklist, and a credential kept somewhere sensible.

Questions

The repository, the handout, and the workflow file are yours to keep.

TAKE IT FURTHER

- Add a branch that deploys to a preview folder, so only main publishes for real
- Swap the stored password for a no-secret handshake (OIDC)
- Add a step that clears a CDN cache after upload
- Add a check that has to pass before the deploy runs